

Apache Spark vs Hadoop MapReduce: A Comparative Analysis of Big Data Analytics Performance

Jia Lin Chua
Faculty of Computing
Universiti Teknologi Malaysia
Johor, Malaysia
chujialin@graduate.utm.my

Abstract— The rapid growth of data generated from digital platforms and Internet of Things (IoT) devices has increased the demand for efficient big data analytics frameworks. Traditional systems such as Hadoop MapReduce have been widely used due to its scalability and fault tolerance. However, its disk-based processing model leads to high latency, especially for iterative tasks. This paper analyses Apache Spark as a modern big data analytics platform and compares it with Hadoop MapReduce in terms of architecture and performance. Apache Spark introduces in-memory processing, which significantly reduces computational overhead and improves execution speed for both batch and real-time analytics. A comparative analysis is presented to highlight key differences, along with real-world case studies demonstrating their practical applications. The findings show that while Apache Spark provides excellent performance and flexibility, it requires higher memory resources, whereas Hadoop MapReduce remains suitable for cost-efficient batch processing.

Keywords— *Big Data Analytics, Apache Spark, Hadoop MapReduce, Performance Analysis*

I. INTRODUCTION

The rapid growth of data produced from social media, digital platforms, and Internet of Things (IoT) has led to the emerging of big data analytics in modern computing. As a result, organizations increasingly rely on data-driven insights to enhance their decision-making skills and improve operational efficiency (Gade, 2021). However, selecting an appropriate computational framework is very important, as different technologies perform differently even on same use case.

Conventional big data processing systems such as Hadoop MapReduce was widely adopted in big data analytics field due to its scalability and robustness (Grolinger et al., 2014). Grolinger et al. (2014) state that its distributed architecture and fault tolerance mechanisms make it suitable for processing large datasets in batch environments. However, Hadoop MapReduce's disk I/O operations may cause latency and computational overhead in scenarios involving iterative computations (Diwakar Reddy & Geetha Reddy, 2017).

With the increasing demand for efficient data processing, modern framework such as Apache Spark was introduced to enhance the performance. Apache Spark introduces in-memory processing and supports batch and real-time processing (Ketu et al., 2020). However, the in-memory processing requires a lot of RAM, thus the implementation cost is higher (Urazmatov & Kuzibayev, 2022).

This paper aims to analyse Apache Spark as a big data analytics tool and compare with Hadoop MapReduce in terms of architecture, performance, and usability. The paper is organized as follows: Section II discusses big data analytics concept, Section III explains Apache Spark, Section IV presents Hadoop MapReduce, Section V provides a comparative analysis between two technologies, and Section VI concludes the paper.

II. OVERVIEW OF BIG DATA ANALYTICS

Rath (2022) states that big data analytics refers to the process of exploring large and complex datasets to discover hidden patterns, trends, and valuable insights. It integrates with modern techniques like artificial intelligence and statistical analysis to process unstructured, semi-structured, and structured data. Moreover, the 4Vs, which are the Volume, Velocity, Variety, and Veracity define the characteristics of big data, representing the scale, speed, diversity, and reliability of data (B M K Prasad et al., 2019). These characteristics make big data difficult to analyze directly. Therefore, visualization tools like Tableau and Power BI are used to present insights in a more understandable way during analytical process. Big data analytics is widely applied in domains such as healthcare, business and marketing, and social media. However, integrating data from multiple sources and format remains challenging.

III. APACHE SPARK

Apache Spark is an open-source and distributed processing system designed for big data analytics. It is known for its in-memory caching and optimized query processing, which allows quick analytic queries for all data sizes (Hassan & Bansal, 2022). Besides, Apache Spark supports operations such as batch processing, real-time data streaming, and machine learning within a unified platform. Due to its speed, scalability, and versatility, it has become a leading tool in the big data analytics domain (Salloum et al., 2016).

A. Architecture

According to Salloum et al. (2016), Apache Spark architecture is build up by four main components which are the driver program, cluster manager, executors, and worker nodes. Its architecture is built to handle both batch and stream processing efficiently, utilizing in-memory computation for high performance and scalability.

The driver program act as the coordinator that converts user-defined operations into a directed acyclic graph (DAG) of tasks and schedules the tasks for execution on the worker node. In short, it manages the execution of tasks and tracks the progress of the tasks.

The cluster manager is responsible for resource allocation across the cluster. Spark supports multiple cluster managers, including the standalone cluster manager, Apache Mesos, Hadoop YARN, and Kubernetes.

The executors are the Java Virtual Machine (JVM) processes on worker nodes that execute tasks assigned by the driver program. They store data partitions in memory for quick access and perform computations.

The worker nodes are the location where the actual computation happens. Each worker node runs executors.

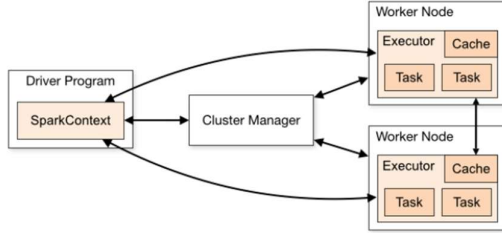


Figure 1: Key Components in Apache Spark (*Cluster Mode Overview - Spark 4.1.1 Documentation*, n.d.)

B. Key Features

Apache Spark provides several important features, including in-memory processing and fault tolerance through lineage.

In-memory processing in Apache Spark refers to storing data in the system's Random Access Memory (RAM) rather than on slower disk drives. With this feature, the time required for read and write data from disk is significantly reduced, thus enhance the system performance. Applications that requires real-time data analysis and iterative computations can be benefited from this feature.

The fault tolerance in Apache Spark is achieved through lineage graphs that tracks the sequence of transformation applied to Resilient Distributed Datasets (RDDs). If a partition is lost, Spark can recompute it by retracing the transformations, thus preventing data lost.

C. Real-World Case Study

Salloum et al. (2016) states that ING Bank use Apache Spark for real-time anomaly detection. Apache Spark's distributed streaming capabilities was deployed to continuously monitor transactions and system behaviors, and identify irregular patterns immediately. This enhances the fraud prevention, operational resilience, and reduces potential risks in financial environment.

D. Advantages and Limitations

The advantage of Apache Spark is that it supports diverse workloads, including batch processing and real-time streaming, making it a general-purpose framework. Besides, its in-memory processing reduces the time needed for read and write from disks.

Apart from the benefits, Apache Spark is hard to be customized due to its complex architecture and the need of deep understanding about the architecture. In addition, Apache Spark's in-memory computation requires powerful RAM, causing the high memory consumption and high implementation cost.

IV. HADOOP MAPREDUCE

Hadoop MapReduce is a programming framework designed for processing and analysing large-scale datasets in a distributed and parallel manner (Pandey et al., 2017). It operates on the Hadoop Distributed File System (HDFS) and is widely used in big data applications such as machine learning and scientific simulations.

A. Architecture

Hadoop MapReduce consists of two main components which are the MapReduce programming model and Hadoop Distributed File System (HDFS). Its framework is designed to process large-scale data across clusters using HDFS.

Data is divided into blocks and processed in parallel through two main phases which are map phase and reduce phase. The map phase converts input data into intermediate key-value pairs, while the reduce phase aggregates these results into final outputs.

A master node manages task scheduling, and slave nodes execute processing tasks. Although this architecture provides scalability and fault tolerance, its reliance on disk-based operations can reduce performance efficiency (Manjula & Meenakshi Sundaram, 2020).

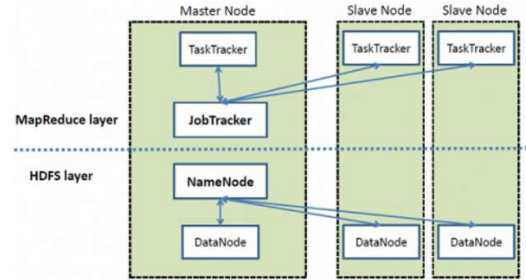


Figure 2: Hadoop MapReduce Architecture (Manjula & Meenakshi Sundaram, 2020)

B. Key Features

The key features of Hadoop MapReduce includes parallel processing and fault tolerance through task re-execution.

Hadoop MapReduce allows parallel processing by dividing large tasks into small and independent tasks that can be executed concurrently through the map phase and reduce phase.

The fault tolerance of Hadoop MapReduce is performed through task re-execution. If any running nodes fails, the failed task is re-executed on another node.

C. Real-World Case Study

Amazon adopts Hadoop MapReduce through its cloud service Amazon Elastic MapReduce (EMR) to process large datasets efficiently. By distributing workloads across EC2 clusters, MapReduce supports large-scale log analysis, clickstream tracking, and retail transaction processing. This batch approach enhances scalability, reliability, and cost-effectiveness, enabling Amazon to discover customer insights, optimize inventory, and strengthen recommendation systems within its big data ecosystem (Daneshyar & Razmjoo, 2012).

D. Advantages and Limitations

The advantage of Hadoop MapReduce is that it is efficient in batch jobs because it distributes tasks across nodes and process them in parallel, thus allows quick processing (Gu et al., 2012). Moreover, Hadoop MapReduce is cost-effective as it uses commodity hardware and disk-based storage to process massive datasets, thus reduces the hardware implementation cost (Maheswari & Sivagami, 2016).

On the other hand, Hadoop MapReduce faces issue such as high latency and inefficiency with dynamic data. Since Hadoop MapReduce involves multiple disk I/O operations, thus causing high latency. Besides, Hadoop MapReduce lacks mechanism that handles dynamic data, therefore it shows poor performance when working with dynamic data.

V. COMPARATIVE ANALYSIS

In this section, comparison between Apache Spark and Hadoop MapReduce is performed.

A. Comparison Table

TABLE I. COMPARISON TABLE BETWEEN APACHE SPARK AND HADOOP MAPREDUCE

Criteria	Hadoop MapReduce	Apache Spark
Processing Model	Disk-based	In-memory
Processing Speed	Slower	Faster
Data Processing	Batch	Batch + real-time
Performance with Iterative Workloads	Low	High
Fault Tolerance	Task re-execution	Lineage-based
Memory Required (RAM)	Lower	Higher
Best Use Case	Non-dynamic, large-scale batch processing jobs	Batch processing, real-time streaming jobs

The comparison between Hadoop MapReduce and Apache Spark highlights several key differences in their performance and design. Hadoop MapReduce operates on a disk-based processing model, which makes it slower but reliable for large-scale batch processing jobs. In contrast, Apache Spark uses in-memory computation, which enables faster processing and supports both batch and real-time data processing.

When dealing with iterative workloads, Apache Spark performs better than Hadoop MapReduce because its in-memory model avoids repeating disk I/O, whereas Hadoop MapReduce requires frequent I/O operations to perform such task. Besides, both technologies demonstrate fault tolerance through different method. Hadoop MapReduce achieves reliability through task re-execution, while Apache Spark uses a more efficient lineage-based approach that restores lost data without restarting the failed tasks.

In terms of resource requirements, Hadoop MapReduce requires lower RAM, making it cost-effective for organizations with limited infrastructure. On the other hand, Apache Spark requires higher memory capacity but provides faster processing speed. As a result, Hadoop MapReduce is more suitable for non-dynamic, large-scale batch processing jobs, while Spark is ideal for both batch analytics and real-time streaming applications.

B. Choosing Suitable Technology

From an organizational perspective, the choice between Hadoop MapReduce and Apache Spark depends on specific operational needs. Organizations that focus on large-scale batch processing, such as log analysis, and retail transaction processing, and operate under budget constraints, may benefit more from adopting Hadoop MapReduce due to its cost efficiency and stability (Daneshyar & Razmjoo, 2012). In contrast, organizations that require real-time analytics, fast data processing, or iterative computations, such as those who work in finance, e-commerce, or streaming services, are more likely to benefit from Apache Spark regardless of its high resource requirements (Salloum et al., 2016).

VI. CONCLUSION

In conclusion, both Apache Spark and Hadoop MapReduce play important roles in the field of big data analytics, each offering unique strengths and trade-offs.

Apache Spark provides high performance through its in-memory processing capabilities, making it suitable for real-time analytics, iterative computations, and advanced data processing tasks. However, these advantages come with increased memory requirements and higher implementation costs.

In contrast, Hadoop MapReduce remains a reliable and cost-effective solution for large-scale batch processing. Its disk-based architecture, fault tolerance, and ability to run on commodity hardware makes it a good choice for organizations that prioritize stability and budget efficiency.

To sum up, the selection of a suitable framework should depend on specific use cases and operational constraints.

REFERENCES

- Cluster Mode Overview - Spark 4.1.1 Documentation. (n.d.). <https://spark.apache.org/docs/latest/cluster-overview.html>
- Daneshyar, S., & Razmjoo, M. (2012). Large-scale data processing using mapreduce in cloud computing environment. *International Journal on Web Service Computing (IJWSC)*, 3(4).
- Diwakar Reddy, A., & Geetha Reddy, J. (2017). Algorithms for Iterative Applications in MapReduce Framework. *Advances in Intelligent Systems and Computing*, 51–61. https://doi.org/10.1007/978-981-10-5272-9_5
- Gade, K. R. (2021). Data-Driven Decision Making in a Complex World. *Journal of Computational Innovation*, 1(1).
- Grolinger, K., Hayes, M., Higashino, W. A., L'heureux, A., Allison, D. S., & Capretz, M. A. (2014, June). Challenges for mapreduce in big data. In *2014 IEEE world congress on services* (pp. 182-189). IEEE.
- Gu, R., Yan, J., Yang, X., Yuan, C., & Huang, Y. (2012). Performance Optimization for Short MapReduce Job Execution in Hadoop. *Journal of Computer Research and Development*, 3, 688–694. <https://doi.org/10.1109/cgc.2012.40>
- Hassan, M., & Bansal, S. (2023). S3QLRDF: distributed SPARQL query processing using Apache Spark—a

- comparative performance study. *Distributed and Parallel Databases*, 41(3), 191-231.
- Ketu, S., Kumar Mishra, P., & Agarwal, S. (2020). Performance Analysis of Distributed Computing Frameworks for Big Data Analytics: Hadoop Vs Spark. *Computación Y Sistemas*, 24(2). <https://doi.org/10.13053/cys-24-2-3401>
- Maheswari, N., & Sivagami, M. (2016). Large-scale data analytics tools: apache hive, pig, and hbase. In *Data Science and Big Data Computing: Frameworks and Methodologies* (pp. 191-220). Cham: Springer International Publishing.
- Manjula, K., & Meenakshi Sundaram, S. (2020). Optimized Approach (SPCA) for Load Balancing in Distributed HDFS Cluster. *SN Computer Science*, 1(2). <https://doi.org/10.1007/s42979-020-0107-8>
- Pandey, S., Supriya, M., & Shrivastava, A. (2017). Aggregation algorithm to overcome data travel cost in MapReduce. *2017 International Conference on Computational Intelligence in Data Science (ICCIDIS)*, 1-6. <https://doi.org/10.1109/iccids.2017.8272639>
- Prasad, B. M. K., Singh, K., Pandey, S. S., & O'Kennedy, R. (Eds.). (2019). *Communication and Computing Systems: Proceedings of the 2nd International Conference on Communication and Computing Systems (ICCCS 2018), December 1-2, 2018, Gurgaon, India*. CRC Press.
- Rath, M. (2022). Big Data Analytics and IoT in Smart City Applications. *Research Anthology on Big Data Analytics, Architectures, and Applications*, 364-380. <https://doi.org/10.4018/978-1-6684-3662-2.ch017>
- Salloum, S., Dautov, R., Chen, X., Peng, P. X., & Huang, J. Z. (2016). Big data analytics on Apache Spark. *International Journal of Data Science and Analytics*, 1(3-4), 145-164. <https://doi.org/10.1007/s41060-016-0027-9>
- Urazmatov, T. Q., & Kuzibayev, X. S. (2022). MapReduce and Apache spark: technology analysis, advantages and disadvantages. *Journal of Physics Conference Series*, 2373(5), 052008. <https://doi.org/10.1088/1742-6596/2373/5/052008>

Logbook

Student Name: Chua Jia Lin

Matric no: A23CS0069

Date	Activity	Keyword	AI Prompt Used
9 April 2026	Request ChatGPT to suggest title and body contents of the paper.	None	“Suggest title for topic ‘Big Data Analytics’, include comparison between two platforms”, “Suggest the structure and contents of the paper”
9 April 2026	Find related paper and read the papers to gain understandings about the platform chosen.	“Big Data Analytics”, “Apache Spark”, “Hadoop MapReduce”	“Suggest papers related to Big Data Analytics, Apache Spark, and Hadoop MapReduce”
10 April 2026	Construct Introduction part.	None	“Brief introduction based to the title of paper”
10 April 2026	Construct Overview of Big Data Analytics part.	“Big Data Analytics”	“Big Data Analytics Overview”
11 April 2026	Construct Apache Spark part including Architecture, Key Features, Real-World Case Study, Advantages and Limitations.	“Apache Spark”, “Apache Spark Architecture”, “Apache Spark Features”, “Apache Spark advantages and limitations”	“Apache Spark”, “Apache Spark Architecture”, “Apache Spark Features”, “Apache Spark real-world case study in big data analytics”, “Apache Spark advantages and limitations”
12 April 2026	Construct Hadoop MapReduce part including Architecture, Key	“Hadoop MapReduce”, “Hadoop MapReduce Architecture”, “Hadoop	“Hadoop MapReduce”, “Hadoop MapReduce Architecture”, “Hadoop MapReduce Features”, “Hadoop MapReduce real-world case study in big

	Features, Real-World Case Study, Advantages and Limitations.	MapReduce Features”, “Hadoop MapReduce advantages and limitations”	data analytics”, “Hadoop MapReduce advantages and limitations”
13 April 2026	Construct Comparison Table between Apache Spark and Hadoop MapReduce and discuss the differences.	None	“Suggest criteria for comparison between Apache Spark and Hadoop MapReduce”
14 April 2026	Construct Conclusion and Abstract part.	None	“Summarise the paper”
14 April 2026	Format the paper.	None	None

